PRD — Système de gestion de présence du personnel et fiche personnel

Version : 1.0
Type : Application web
Stack : ReactJs + ExpressJs,
Utilisateurs : Personnel administratif + administrateurs/responsables
Projet : Gestion informatisée des fiches et de la présence des personnels

---

1. Présentation du projet

Le projet consiste à développer une application web permettant de **digitaliser la gestion des fiches et la présence des personnels**.

L'identification des agents se fait à l'aide d'un **QR Code individuel**. Lorsqu'un agent scanne son QR Code à son arrivée, le système l'identifie automatiquement et enregistre sa présence avec la date et l'heure.

Les informations collectées permettent ensuite de générer automatiquement les **fiches de présence**, de suivre les retards et absences et de consulter l'historique des présences.

L'objectif est de remplacer ou de compléter le processus manuel de remplissage des fiches de présence par un système informatisé.

---

2. Objectifs

Objectif général

Mettre en place un système permettant d'**automatiser l'enregistrement, le suivi et la génération des fiches de présence du personnel**.

Objectifs spécifiques

Le système devra permettre de :

- identifier un agent grâce à un QR Code ;
- enregistrer automatiquement son heure d'arrivée ;
- enregistrer son heure de départ ;
- déterminer son statut : présent, retard, absent ;

- générer automatiquement une fiche de présence ;
- mettre à jour la fiche après chaque scan ;
- consulter les présences par date ;
- consulter l'historique ;
- rechercher et filtrer les agents ;
- générer/exporter les fiches en PDF ;
- faciliter le contrôle des présences par les responsables.

---

3. Problématique

La gestion traditionnelle des présences peut présenter plusieurs problèmes :

- remplissage manuel des fiches ;
- erreurs de saisie ;
- perte ou détérioration des documents ;
- difficulté à retrouver une ancienne fiche ;
- calcul manuel des retards et absences ;
- perte de temps lors de la compilation des données ;
- difficulté à produire rapidement des statistiques.

Le système proposé vise donc à **centraliser et automatiser ces opérations**.

---

4. Utilisateurs du système

Coordinateur

L'administrateur possède les droits de gestion du système.

Il peut :

- gérer les agents ;
- générer les QR Codes ;
- consulter les présences ;
- modifier certaines informations ;
- consulter l'historique ;
- générer les fiches ;
- exporter les données ;
- gérer les paramètres du système.

Secrétaire

Le secrétaire peut principalement :

- consulter les présences ;
- consulter les absences ;
- consulter les retards ;
- générer les fiches ;
- consulter l'historique ;
- exporter les rapports.
- scanner les code

Agent

L'agent possède un QR Code individuel.

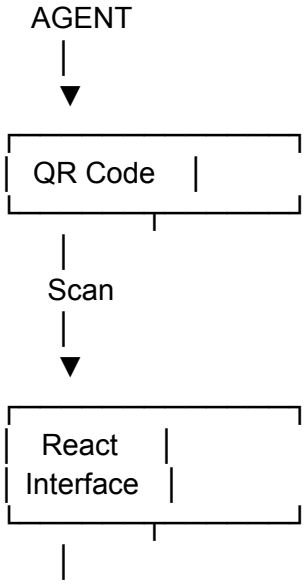
Son interaction avec le système est principalement :

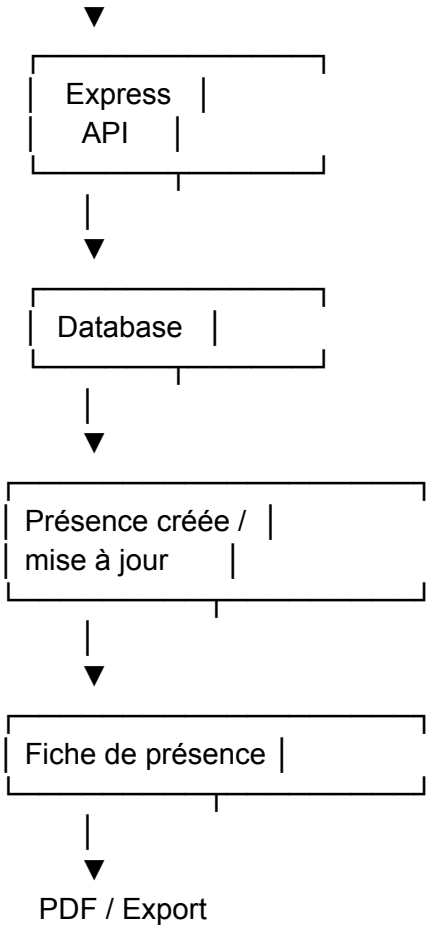
Présence
↓
Scan QR Code
↓
Identification
↓
Enregistrement automatique

---

5. Fonctionnement général

Le workflow principal sera :





---

6. Fonctionnalités

6.1 Authentification

L'application devra disposer d'un système de connexion.

Email / Identifiant
Mot de passe
↓
Authentification
↓
Dashboard

Les rôles pourront être :

Coordonateur
Secrétaire

---

7. Gestion des QR Codes

Chaque agent actif possède un QR Code unique.

Exemple conceptuel :

AGT-001

Le QR Code ne devrait pas contenir directement toutes les informations personnelles.

Il peut contenir un identifiant ou un token :

AGT-001-X8F92A

Le serveur utilise ensuite cet identifiant pour retrouver l'agent.

L'administrateur pourra :

- générer un QR Code ;
- afficher le QR Code ;
- imprimer le QR Code ;
- régénérer un QR Code si nécessaire.

---

8. Enregistrement d'une présence

Lorsqu'un agent scanne son QR Code :

QR Code
↓
Identification agent
↓
Date actuelle
↓
Heure actuelle
↓
Vérification
↓
Enregistrement

Exemple :

Agent : Jean Rakoto
Date : 09/09/2026
Heure : 07:56:21
Type : ARRIVÉE
Statut : PRÉSENT

---

9. Gestion du retard

Une heure de référence sera configurée.

Exemple :

Heure normale : 08:00
Tolérance : 10 minutes

Donc :

07:55 → Présent
08:05 → Présent
08:12 → Retard

Le système déterminera automatiquement le statut.

---

10. Gestion du départ

Le même système peut être utilisé pour l'heure de départ.

Exemple :

Premier scan
↓
ARRIVÉE

Deuxième scan
↓
DÉPART

La fiche devient :

Jean Rakoto

Arrivée : 07:56
Départ : 16:04
Statut : Présent

Le système devra empêcher les incohérences comme plusieurs arrivées successives sans départ.

---

11. Gestion des absences

À une heure définie, le système peut déterminer les agents qui ne sont pas enregistrés comme présents.

Exemple :

09:00

38 agents attendus

35 présents
2 retards
1 absent

Le secrétaire et/ou le coordinateur pourra également corriger manuellement une situation particulière si nécessaire.

---

12. Fiche de présence : Dillane

Le système doit générer automatiquement une fiche à partir des données enregistrées.

Exemple :

DIRECTION RÉGIONALE
DES FINANCES ET DU BUDGET

FICHE DE PRÉSENCE

Date : 09/09/2026

N°	Agent	Arrivée	Départ	Statut

01	Jean Rakoto	07:56	16:04	Présent
02	Marie R.	08:17	16:02	Retard
03	Paul R.	—	—	Absent

La fiche pourra être :

- visualisée dans React ;
- générée en PDF ;
- imprimée ;
- consultée ultérieurement.

---

13. Historique

Le système conservera les anciennes présences,.

Exemple :

09/09/2026
08/09/2026
07/09/202
06/09/2026
...

L'utilisateur pourra sélectionner une date pour retrouver la fiche correspondante.

Filtres :

Date
Agent
Service
Statut
Période

---

14. Tableau de bord

Le dashboard pourra afficher :

Agents : 48

┌──────────────────┐
│ Présents : 42 │
└──────────────────┘

┌──────────────────┐
│ Retards : 4 │
└──────────────────┘

┌──────────────────┐
│ Absents : 2 │
└──────────────────┘

Avec éventuellement un graphique :

Présence cette semaine

Lun ████████████
Mar ██████████
Mer ████████████
Jeu ████████
Ven ████████████

---

15. Architecture technique

Frontend

React

Responsable de :

- interface utilisateur ;
- dashboard ;
- gestion des agents ;
- affichage des présences ;
- interface de scan ;
- affichage des fiches ;
- filtres ;
- historique.

Backend

Express.js / Node.js

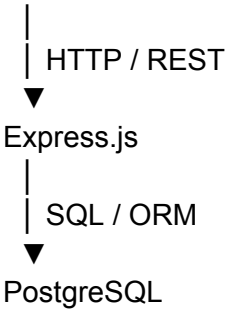
Responsable de :

- API REST ;
- authentification ;
- logique métier ;
- gestion des présences ;
- validation des scans ;
- génération des données nécessaires aux fiches ;
- communication avec la base de données.

Base de données

PostgreSQL .

Architecture :



(Vous pourrez utiliser **Prisma** côté Express pour simplifier l'accès à PostgreSQL.)

Développement

Git
Github
Visual Studio Code

---

18. MVP

✓ Connexion
✓ Gestion agents
✓ Génération QR
✓ Scan QR
✓ Enregistrement arrivée
✓ Enregistrement départ
✓ Détection retard
✓ Gestion absence
✓ Tableau de présence
✓ Génération fiche

✓ Remplissage automatique
✓ Historique
✓ Export PDF

QR Code → présence → base de données → fiche automatiquement remplie → historique → PDF.